SAM DIRR • OCTOBER 11, 2026 • CSD 370 - ORM ASSIGNMENT

Operational Risk Management During Software Deployment

Research comparison and recommendation for Mesusa Corporation

DEPLOYMENT RISK RESEARCH BRIEF

Prepared for Moira Bennie, Chief Information Officer

Prepared by George Everett, Senior Developer

RESEARCH QUESTION

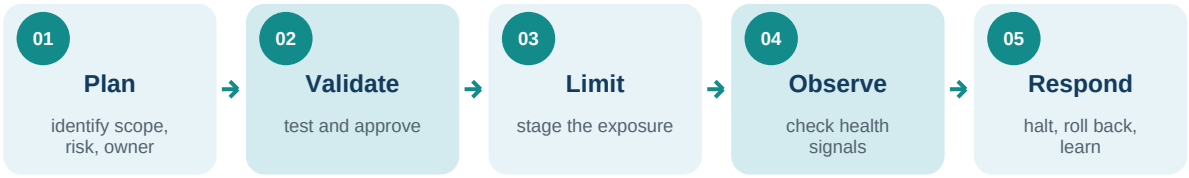
How should MeCo manage operational risk during a deployment?

Operational risk during deployment is the possibility that a software, configuration, infrastructure, or data change will interrupt service, weaken a security control, corrupt data, or delay recovery. The most effective approach does not depend on a single product. It combines change control, tested release practices, limited exposure, monitoring, and a proven recovery path.

This brief evaluates three authoritative sources against the same questions: What deployment risk does the source address? Does it provide usable steps? Does it provide practices that reduce risk?

Source	Primary contribution	Does it provide steps?
NIST SP 800-128	Governance for secure configuration management and change control.	**Yes.** A structured SecCM process, but not a detailed release-engineering playbook.
Microsoft Safe Deployment Practices	End-to-end deployment safety: progressive rollout, health checks, recovery, guardrails, and emergencies.	**Yes.** It lays out implementation principles and concrete safety controls.
AWS Mitigate Deployment Risks	A direct operational action plan for failed-change planning, testing, safe rollout, and automated rollback.	**Yes.** Four explicit practices with implementation guidance.

The shared deployment-risk cycle



All three sources support a controlled, evidence-based cycle rather than an all-at-once production release.

Figure 1. A synthesis of the controls the sources place around a production change.

Research conclusion

The sources agree on a simple principle: production deployment is a risk-management activity. A team should never rely on an all-at-once release with no proof of health or recovery. The later pages examine how each source turns that principle into practice.

SOURCE ONE

NIST SP 800-128

Guide for Security-Focused Configuration Management of Information Systems presents configuration management as a security and risk-management discipline. The source is broader than deployment engineering: it focuses on managing and monitoring configurations so an organization can maintain adequate security while supporting business services (Johnson et al., 2011). For MeCo, the value is that every deployment changes a configuration baseline and therefore needs traceability, security analysis, and verification.

Question	Assessment
What deployment risk does it address?	Unauthorized, undocumented, insecure, or poorly understood configuration changes that weaken a system's security posture or make recovery difficult.
Does it provide steps?	**Yes.** It describes planning the security-focused CM program, identifying configuration items and baselines, controlling changes, monitoring configuration, and verifying/auditing the result. The source provides a governance process rather than a minute-by-minute deployment procedure.
Does it provide risk-reduction practices?	**Yes.** It emphasizes approved baselines, security impact analysis before change, documentation of approved and implemented changes, status accounting, least-privilege change access, and configuration monitoring.
Main strength	Vendor-neutral and directly aligned with MeCo's future change control/configuration management process. It makes ownership, evidence, and system state visible.
Main limitation	It predates many common DevOps patterns and gives less detail on canary releases, health-based promotion, feature flags, and automated rollback.

What MeCo can use immediately

Treat source code, deployment artifacts, infrastructure-as-code, application settings, database schemas, feature-flag states, service accounts, and runbooks as configuration items. For every meaningful deployment, link the change request to affected items, record the approved baseline, and capture test, approval, and deployment evidence. That creates a reliable starting point for rollback, audit, and incident response.

Practical NIST-aligned control	Deployment effect
Baseline and versioned artifact	Makes the prior known-good state identifiable and recoverable.
Security impact analysis	Surfaces changed permissions, exposed interfaces, data flows, and dependencies before release.
Configuration monitoring	Detects drift between the approved deployment and the actual environment.

SOURCE TWO

Microsoft Safe Deployment Practices

Microsoft's **Architecture strategies for safe deployment practices** treats every production change - application code, infrastructure as code, configuration, and feature flags - as risky. It provides a practical operational model for reducing human error and limiting the customer impact of a bad deployment (Microsoft, n.d.). Its advice is relevant beyond Azure because the core controls are process and engineering practices, not Azure products.

Question	Assessment
What deployment risk does it address?	Human error, large blast radius, weak visibility, unsafe pipeline practices, and delayed recovery from a bad release.
Does it provide steps?	**Yes.** It organizes implementation around safety and consistency, progressive exposure, health models, failure detection, deployment guardrails, emergency protocols, and safe decommissioning.
Does it provide risk-reduction practices?	**Yes.** It recommends smaller frequent changes; production-like staging; code, security, and compliance checks; canary or blue-green deployment; bake time; artifact versioning; feature flags; circuit breakers; and halt/rollback or roll-forward procedures.
Main strength	The most complete operational deployment guide in this comparison. It covers normal, emergency, and retirement scenarios, and explicitly joins deployment phases to health signals and recovery.
Main limitation	Some named tools and examples are Azure-specific. Stateful changes, such as databases and schemas, require extra rollback planning.

A usable deployment sequence

Before production	During progressive rollout	If health degrades
Review code; scan for security and compliance issues; validate in a production-like environment; version the artifact; confirm a rollback plan.	Expose a small user group or traffic share; observe technical and user-facing health signals; use a meaningful bake time; promote only when gates pass.	Stop promotion immediately. Investigate the impact. Roll back to the known-good configuration, roll forward with a safe fix, or redeploy the known-good infrastructure.

SOURCE THREE

AWS Well-Architected Mitigate Deployment Risks

AWS's Operational Excellence guidance provides the clearest task sequence of the three sources. It frames deployment risk around quality feedback and rapid recovery, then divides mitigation into four practices: plan for unsuccessful changes, test deployments, use safe rollout strategies, and automate testing and rollback (Amazon Web Services [AWS], n.d.). The named AWS tools are optional; the operating model is portable.

Question	Assessment
What deployment risk does it address?	Customer impact and prolonged recovery when a failed change reaches production without a tested exit path.
Does it provide steps?	**Yes.** The source provides four explicit practices and linked guidance for documented recovery policies, production-like testing, staged strategies, quality criteria, automation, monitoring, and postdeployment tests.
Does it provide risk-reduction practices?	**Yes.** It recommends smaller releases, documented rollback/fix-forward policy, change visibility, drift checks, approval gates, feature flags, canary or blue-green releases, monitoring, and automatic reversion under predetermined failure conditions.
Main strength	Most direct step-by-step implementation path. The source also identifies the risks and effort of missing practices, which helps MeCo prioritize implementation.
Main limitation	Many examples use AWS services, and full production-like testing can require substantial engineering maturity and effort.

Four practices MeCo can adapt

1. Plan for failure Document rollback or fix-forward ownership, criteria, and the last known-good state.	**2. Test the deployment** Use representative configuration, security controls, functional tests, integration tests, and health checks.
3. Use a safe rollout Use feature flags, canary, traffic splitting, or blue-green deployment rather than immediate full exposure.	**4. Automate recovery** Define failure thresholds, integrate testing and monitoring into CI/CD, and trigger rollback when conditions are met.

Why the steps matter

A documented but manual recovery plan is better than no plan, but automation reduces the time in which users are exposed to a known bad change. AWS therefore reinforces the control point that is easy to miss: deployment risk includes the time required to detect and reverse the problem, not only the likelihood of introducing one.

SYNTHESIS AND RECOMMENDATION

The sources agree on the controls that matter

Shared theme	What all three sources say
Controlled change	Identify the change, owner, target state, and risk before it reaches production.
Predeployment validation	Use review, tests, security checks, and a production-like setting to catch preventable errors before customer exposure.
Limited exposure	Avoid an immediate all-at-once rollout. Use a small, staged population or a parallel environment to limit blast radius.
Observable health	Use technical and user-impact signals to decide whether to promote, pause, or recover.
Known recovery path	Keep a known-good state and define rollback or fix-forward action before deployment.
Learning loop	Document results, examine failed changes, and improve controls, templates, and deployment design.

Recommended source for MeCo

If MeCo must begin with one source, it should use Microsoft's Safe Deployment Practices as its operational playbook. It gives the most complete deployment sequence: small changes, predeployment gates, progressive exposure, health models, failure detection, recovery, emergency procedures, and safe retirement. It applies to code, infrastructure, configuration, and feature flags, so it fits MeCo's deployment-risk question closely.

That recommendation has a qualification. Microsoft's tool examples are Azure-specific, and the guide alone does not replace MeCo's configuration-management policy. MeCo should therefore use NIST SP 800-128 for governance and evidence, then translate Microsoft's techniques into its existing tools. AWS is a valuable implementation checklist, especially for automation and rollback criteria, but Microsoft supplies the broadest day-to-day operating model.

Immediate action plan for MeCo

First 30 days	Next 30 days	Then
Name an owner; classify changes; set required evidence; establish a known-good baseline; write rollback/fix-forward criteria.	Add predeployment reviews and automated checks; pilot a staging environment; identify 3-5 health signals and stop conditions.	Pilot canary or blue-green rollout for one service; measure recovery time; review every failed or emergency deployment.

Bottom line

A safe deployment is not merely a successful release. It is a controlled change that can be observed, halted, recovered, and explained. Combining NIST governance with a modern safe-deployment routine gives MeCo a practical ORM approach that protects customers without preventing regular delivery.

SOURCES CONSULTED

References

Amazon Web Services. (n.d.). *Mitigate deployment risks*. AWS Well-Architected Framework. Retrieved October 11, 2026, from https://docs.aws.amazon.com/wellarchitected/latest/operational-excellence-pillar/mitigate-deployment-risks.html

Johnson, A., Dempsey, K., Ross, R., Gupta, S., & Bailey, D. (2011). *Guide for security-focused configuration management of information systems* (NIST Special Publication 800-128). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-128

Microsoft. (n.d.). *Architecture strategies for safe deployment practices*. Azure Well-Architected Framework. Retrieved October 11, 2026, from https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/safe-deployments

AI assistance disclosure. OpenAI Codex was used to help locate sources, organize and revise the research comparison, and create the visual layout and graphics. The author reviewed and approved the final content.